INLAcircular: User Guide and Function Reference

Xiang Ye

Contents

1 Scope and conventions **2**

2 Installation **2**

3 Circular distributions **3**
3.1 Quick comparison . . . 3
3.2 Von Mises . . . 3
3.3 Link-adjusted von Mises . . . 4
3.4 Cardioid and wrapped Cauchy densities . . . 4

4 Penalized-complexity priors **5**
4.1 Von Mises prior with uniform base: pc.vm0 . . . 6
4.2 Von Mises prior with point-mass base: pc.vminf . . . 6
4.3 Cardioid prior with uniform base: pc.card0 . . . 7
4.4 Cardioid prior with base kappa = 1/2: pc.card . . . 7
4.5 Wrapped Cauchy prior: pc.wc . . . 8
4.6 Density, probability, and vectorization rules . . . 9

5 Direct fitting with R-INLA **9**
5.1 LAvM likelihood controls . . . 10
5.2 Convenience call for a single LAvM model . . . 11

6 Building model specifications **11**
6.1 Formula terms and their interpretation . . . 12
6.2 Mapping observations with `index()` . . . 13

7 Fitting with `inlacc()` **14**
7.1 Arguments and defaults . . . 14
7.2 Single-likelihood fit . . . 15
7.3 Joint likelihoods and copied predictors . . . 15
7.4 Returned object . . . 16

8 Modelling multivariate effects with `graphpcor` **16**
8.1 Declaring an LKJ-correlated effect . . . 16
8.2 Replicate indices . . . 17
8.3 Recovering covariance information . . . 18

9 Results and summaries **19**

10 Utilities and included data **19**
10.1 Modified Bessel function . . . 19
10.2 New York wind data . . . 20

11 Complete public API index **20**

12 Numerical implementation notes **20**

13 Reproducibility **20**



1 Scope and conventions

INLAcircular provides probability functions and a model-building interface for Bayesian circular and joint circular regression with Integrated Nested Laplace Approximation (INLA). Its core numerical routines are written in C and exposed through ordinary R functions.

The package follows these conventions:

- Angles are measured in radians.
- Location parameters are named mu.
- Concentration parameters are named kappa, including for cardioid and wrapped Cauchy distributions.
- Density functions use log = TRUE for log-densities.
- PC-prior distribution and quantile functions use log.p = TRUE for probabilities on the log scale.
- The d/p/q/r prefix has its usual R meaning: density, distribution function, quantile function, and random generation.

The core distribution and PC-prior functions can be used without `INLA`. Model fitting requires the optional `INLA` package, and LKJ multivariate random effects also require `graphpcor`.

2 Installation

After the package is published on CRAN, install it with:

```
install.packages("INLAcircular")
```

The installation above provides the circular distributions and PC-prior functions without installing INLA. For model fitting, install optional INLA >= 25.08.21 from the current stable repository separately:

```
install.packages(
  "INLA",
  repos = c(
    getOption("repos"),
    INLA = "https://inla.r-inla-download.org/R/stable"
  )
)
```

For a development installation from GitHub:

```
install.packages("remotes")
remotes::install_github(
```

```
  "XiangYEstats/INLAcircular"
)
```

```
library(INLAcircular)
```

Attaching INLA is optional because `INLAcircular` loads its namespace when a model-fitting function needs it. If both packages are attached and you want the compatibility `inla()` function on the search path, load `INLA` first and then `INLAcircular`.

3 Circular distributions

3.1 Quick comparison

The package supplies the full d/p/q/r interface for the von Mises and link-adjusted von Mises distributions. It supplies compiled densities for the cardioid and wrapped Cauchy distributions.

```
theta <- seq(-pi, pi, length.out = 361)
densities <- cbind(
  "von Mises" = dvm(theta, mu = 0, kappa = 2),
  "cardioid" = dcardioid(theta, mu = 0, kappa = 0.4),
  "wrapped Cauchy" =
    dwrappedcauchy(theta, mu = 0, kappa = 0.7)
)

matplot(
  theta,
  densities,
  type = "l",
  lty = 1,
  xlab = "Angle (radians)",
  ylab = "Density"
)
legend(
  "topright",
  legend = colnames(densities),
  col = seq_len(ncol(densities)),
  lty = 1,
  bty = "n"
)
```

3.2 Von Mises

dvm(), pvm(), qvm(), and rvm() evaluate the von Mises distribution with density

$$f(x \mid \mu, \kappa) = \frac{\exp\{\kappa \cos(x - \mu)\}}{2\pi I_0(\kappa)}.$$

kappa = 0 gives the circular uniform limit. The CDF and quantile functions use a cached spline grid; len controls its resolution. With strategy = "circular", the CDF origin follows the mean direction. The alternative strategy = "linear" retains a fixed linear representation.

```
set.seed(1)
rvm(5, mu = 0, kappa = 3)
#> [1] -0.3859961 -0.2006438  0.1103994  0.8385810 -0.5174967
pvm(pi / 2, mu = 0, kappa = 2)
```

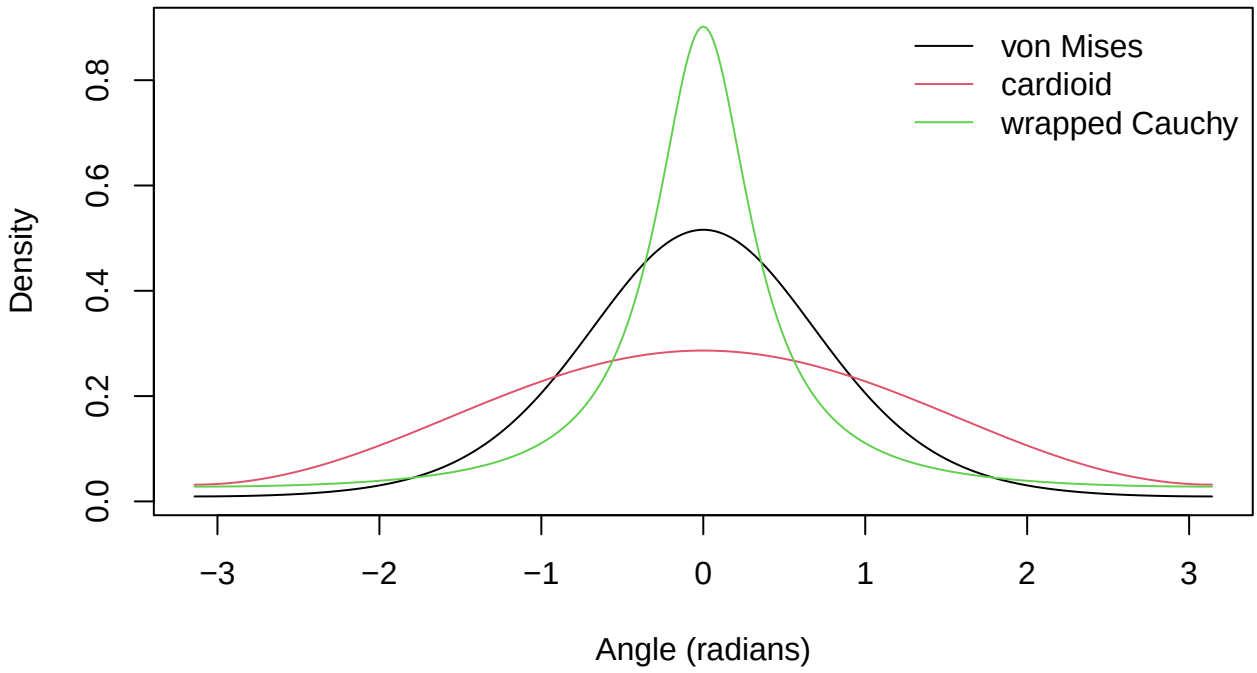


Figure 1: Three circular density families with mean direction zero.

```
#> [1] 0.9625691
qvm(0.95, mu = 0, kappa = 2)
#> [1] 1.416584
```

3.3 Link-adjusted von Mises

dlavm(), plavm(), qlavm(), and rlavm() use a linear predictor eta and concentration kappa. The current direct distribution interface uses the inverse-tangent transformation

$$z = 2\arctan\{\tan(x/2) - \eta\}.$$

lavm.link("tan") returns the corresponding transformation and inverse functions. The link_obj argument in the p/q/r functions is reserved for interface compatibility; the current compiled implementation uses the inverse-tangent link.

```
set.seed(2)
rlavm(5, eta = 1.5, kappa = 3)
#> [1] 1.763773 2.058985 1.998958 1.745235 2.237412
dlavm(c(-1, 0, 1), eta = 1.5, kappa = 3)
#> [1] 0.001291978 0.003164735 0.025559191
plavm(pi / 2, eta = 1.5, kappa = 3)
#> [1] 0.07300838
qlavm(0.5, eta = 1.5, kappa = 3)
#> [1] 1.965116
```

3.4 Cardioid and wrapped Cauchy densities

The cardioid density is

$$f_{\text{card}}(x \mid \mu, \kappa) = \frac{1 + 2\kappa\cos(x - \mu)}{2\pi}, \qquad 0 \le \kappa \le \frac{1}{2}.$$

Use dcardioid(), or its short alias dcard().

The wrapped Cauchy density is

$$f_{\text{wc}}(x \mid \mu, \kappa) = \frac{1 - \kappa^2}{2\pi\{1 + \kappa^2 - 2\kappa\cos(x - \mu)\}}, \qquad 0 \le \kappa < 1.$$

Use dwrappedcauchy(), or its short alias dwc(). The endpoint kappa = 1 is a point-mass limit: the compiled density reports Inf at the mean direction and zero elsewhere. Finite-parameter likelihoods should use kappa < 1.

Both functions recycle x, mu, and kappa to a common length.

```
x <- c(-pi / 2, 0, pi / 2)
dcardioid(x, mu = 0, kappa = c(0, 0.25, 0.5))
#> [1] 0.1591549 0.2387324 0.1591549
dwrappedcauchy(x, mu = 0, kappa = 0.7, log = TRUE)
#> [1] -2.909998 -0.103276 -2.909998
```

4 Penalized-complexity priors

A PC prior assigns an exponential distribution with rate lambda to a distance from a simpler base model. Larger lambda puts more prior mass near the base. All five prior families expose matching density, CDF, quantile, and random-generation functions.

Help topic	Base model	Support for kappa	Special behavior
pc.vm0	von Mises uniform limit, kappa = 0	[0, Inf)	Continuous
pc.vminf	von Mises point-mass limit	[0, Inf)	Atom exp(-lambda) at zero
pc.card0	cardioid uniform model, kappa = 0	[0, 1/2]	Normalized truncated exponential distance
pc.card	cardioid base, kappa = 1/2	[0, 1/2]	Atom at zero; integrable singularity at 1/2
pc.wc	wrapped Cauchy uniform model, kappa = 0	[0, 1)	Continuous; open upper endpoint

For a monotone distance $d(\kappa)$, an untruncated continuous PC prior has density

$$\pi(\kappa) = \lambda \exp\{-\lambda d(\kappa)\} \left| d'(\kappa) \right|.$$

When the attainable distance range is bounded, the remaining exponential tail must be handled explicitly. pc.card0 truncates and normalizes the exponential distance law; pc.vminf and pc.card collapse the remaining tail to a boundary atom. The density functions below return densities with respect to Lebesgue measure. A boundary atom is therefore represented by the CDF and RNG, not added to the value returned by a density function.

4.1 Von Mises prior with uniform base: pc.vm0

Write

$$A(\kappa) = \frac{I_1(\kappa)}{I_0(\kappa)}, \qquad A'(\kappa) = 1 - A(\kappa)^2 - \frac{A(\kappa)}{\kappa},$$

where the continuous limit is $A'(0) = 1/2$. The distance from the circular uniform model is

$$d_0(\kappa) = \sqrt{\kappa A(\kappa) - \log I_0(\kappa)}.$$

For $\kappa > 0$, the density and CDF are

$$\boxed{\pi_0(\kappa) = \lambda e^{-\lambda d_0(\kappa)} \frac{\kappa A'(\kappa)}{2 d_0(\kappa)}}, \qquad F_0(\kappa) = 1 - e^{-\lambda d_0(\kappa)}.$$

The density has the finite endpoint limit $\pi_0(0) = \lambda/2$. This is a continuous distribution with no atom at zero. Use dpc.vm0(), ppc.vm0(), qpc.vm0(), and rpc.vm0().

```
probability <- c(0.1, 0.5, 0.9)
q0 <- qpc.vm0(probability, lambda = 2)
cbind(
  probability = probability,
  quantile = q0,
  recovered = ppc.vm0(q0, lambda = 2)
)
#>      probability  quantile recovered
#> [1,]         0.1 0.1054703       0.1
#> [2,]         0.5 0.7266853       0.5
#> [3,]         0.9 6.6791614       0.9

dpc.vm0(c(0, 0.5, 2), lambda = 2)
#> [1] 1.00000000 0.57267102 0.09577906
```

4.2 Von Mises prior with point-mass base: pc.vminf

The distance from the point-mass limit $\kappa \to \infty$ is

$$d_\infty(\kappa) = \sqrt{1 - A(\kappa)}.$$

The continuous density for $\kappa > 0$ and the full CDF are

$$\boxed{\pi_\infty^{\text{cont}}(\kappa) = \lambda e^{-\lambda d_\infty(\kappa)} \frac{A'(\kappa)}{2 d_\infty(\kappa)}}, \qquad F_\infty(\kappa) = e^{-\lambda d_\infty(\kappa)}.$$

Because $d_\infty(0) = 1$, the full law has boundary mass $\Pr(\kappa = 0) = e^{-\lambda}$. dpc.vminf() returns only the continuous component, whose right-hand endpoint limit is $\lambda e^{-\lambda}/4$. ppc.vminf(), qpc.vminf(), and rpc.vminf() include the atom.

```
lambda <- 2
atom_vm <- exp(-lambda)
c(computed = ppc.vminf(0, lambda), expected = atom_vm)
#>  computed  expected
```

```
#> 0.1353353 0.1353353
dpc.vminf(c(0, 0.5, 2), lambda)
#> [1] 0.06766764 0.09193625 0.09948602
qpc.vminf(c(atom_vm / 2, (1 + atom_vm) / 2), lambda)
#> [1] 0.000000 6.527716
```

4.3 Cardioid prior with uniform base: pc.card0

Let

$$s(\kappa) = \sqrt{1 - 4\kappa^2}, \qquad d_0(\kappa) = \sqrt{1 - s(\kappa) + \log\{(1 + s(\kappa))/2\}}.$$

The distance is bounded above by $D_0 = \sqrt{1 - \log 2}$. Therefore pc.card0 uses the normalized, truncated exponential density

$$\boxed{\pi_0(\kappa) = \frac{\lambda e^{-\lambda d_0(\kappa)}}{1 - e^{-\lambda D_0}} \frac{2\kappa}{\{1 + s(\kappa)\} d_0(\kappa)}}.$$

Its CDF is

$$F_0(\kappa) = \frac{1 - \exp\{-\lambda d_0(\kappa)\}}{1 - \exp(-\lambda D_0)}.$$

Use dpc.card0(), ppc.card0(), qpc.card0(), and rpc.card0(). The density at zero is evaluated by continuity and equals $\lambda/\{1 - e^{-\lambda D_0}\}$. At $\kappa = 1/2$, it equals $\lambda e^{-\lambda D_0}/[D_0\{1 - e^{-\lambda D_0}\}]$.

```
probability <- c(0.1, 0.5, 0.9)
quantile <- qpc.card0(probability, lambda = 2)
cbind(
  probability = probability,
  quantile = quantile,
  recovered = ppc.card0(quantile, lambda = 2)
)
#>      probability   quantile recovered
#> [1,]         0.1 0.03465088       0.1
#> [2,]         0.5 0.20172637       0.5
#> [3,]         0.9 0.43368208       0.9
dpc.card0(c(0, 0.25, 0.5), lambda = 2)
#> [1] 2.986217 1.892776 1.780359
```

4.4 Cardioid prior with base kappa = 1/2: pc.card

The distance to the concentrated cardioid base is

$$d_{1/2}(\kappa) = \sqrt{1 - 2\kappa - s(\kappa) + \log\{1 + s(\kappa)\}}.$$

It decreases from $\sqrt{\log 2}$ at zero to zero at the base. Its continuous density is

$$\boxed{\pi_{1/2}^{\text{cont}}(\kappa) = \lambda e^{-\lambda d_{1/2}(\kappa)} \frac{1 + s(\kappa) - 2\kappa}{\{1 + s(\kappa)\} d_{1/2}(\kappa)}}.$$

The full CDF is

$$F_{1/2}(\kappa) = \exp\{-\lambda d_{1/2}(\kappa)\}.$$

Consequently, the distribution has boundary mass $\exp\{-\lambda\sqrt{\log 2}\}$ at kappa = 0. dpc.card() returns the continuous density component. ppc.card(), qpc.card(), and rpc.card() include the atom. The continuous density tends to Inf at kappa = 0.5, but the singularity is integrable. At zero, its continuous component is $\lambda e^{-\lambda\sqrt{\log 2}}/\sqrt{\log 2}$; this value is distinct from the atom probability.

```
lambda <- 2
atom_card <- exp(-lambda * sqrt(log(2)))
c(computed = ppc.card(0, lambda), expected = atom_card)
#> computed expected
#>  0.18917  0.18917
qpc.card(c(atom_card / 2, (1 + atom_card) / 2), lambda)
#> [1] 0.0000000 0.4024443
dpc.card(c(0, 0.25, 0.49), lambda)
#> [1] 0.4544326 1.0445693 6.6278466
```

4.5 Wrapped Cauchy prior: pc.wc

For the wrapped Cauchy family,

$$d_{\mathrm{wc}}(\kappa) = \sqrt{-\log(1-\kappa^2)}$$

maps [0, 1) to the full nonnegative distance axis. Its density is

$$\boxed{\pi_{\mathrm{wc}}(\kappa) = \lambda e^{-\lambda d_{\mathrm{wc}}(\kappa)} \frac{\kappa}{(1-\kappa^2) d_{\mathrm{wc}}(\kappa)}}.$$

The CDF is

$$F_{\mathrm{wc}}(\kappa) = 1 - \exp\{-\lambda d_{\mathrm{wc}}(\kappa)\}.$$

Use dpc.wc(), ppc.wc(), qpc.wc(), and rpc.wc(). The density limit at zero is lambda; the density diverges as kappa approaches one. Interior-probability quantiles (`0 < p < 1`) and RNG output remain strictly below one. There is no atom at one; the value dpc.wc(1, lambda) is the limiting density convention.

```
q_wc <- qpc.wc(c(0.1, 0.5, 0.9), lambda = 2)
cbind(quantile = q_wc, recovered = ppc.wc(q_wc, lambda = 2))
#>         quantile recovered
#> [1,] 0.05264373       0.1
#> [2,] 0.33642236       0.5
#> [3,] 0.85692657       0.9

set.seed(3)
rpc.wc(5, lambda = 2)
#> [1] 0.09179213 0.70196661 0.23947530 0.19660997 0.43736859
dpc.wc(c(0, 0.5, 0.9), lambda = 2)
#> [1] 2.0000000 0.8503661 0.5585013
```

4.6 Density, probability, and vectorization rules

Every family follows the same call pattern:

```
dpc.NAME(kappa, lambda, log = FALSE)
ppc.NAME(q, lambda, log.p = FALSE)
qpc.NAME(p, lambda, log.p = FALSE)
rpc.NAME(n, lambda)
```

The PC-prior functions recycle concentrations or probabilities against lambda. The rate must always contain positive, finite values. With log.p = TRUE, CDFs return log-probabilities and quantiles accept log-probabilities. The density flag is log, and both flags must be one non-missing logical value. There is no lower.tail argument; all probabilities are lower-tail.

Values and lambda recycle to their common maximum length, with a warning for non-multiple lengths. For random generation, a vector n requests length(n) draws; otherwise its scalar value is converted to an integer. The generators use R's RNG stream. Apart from the documented limiting-endpoint conventions, densities are zero outside their mathematical support, while invalid probabilities produce NaN.

Names such as pc.card0 are help topics and prior labels, not callable R functions. Use the corresponding d/p/q/r functions.

```
dpc.card0(c(0.1, 0.2, 0.3), lambda = c(1, 2, 3), log = TRUE)
#> [1] 0.7622848 0.7219704 0.4663980
qpc.wc(log(c(0.1, 0.5, 0.9)), lambda = 2, log.p = TRUE)
#> [1] 0.05264373 0.33642236 0.85692657
```

5 Direct fitting with R-INLA

This section uses INLA::inla() literally. The LAvM likelihood in this package is an external compiled cloglike, so a direct call has three explicit preparation steps:

1. convert the circular response with INLA::inla.mdata();
2. construct the compiled likelihood with lavm.cloglike(); and
3. call INLA::inla() with family = "cloglike".

```
lavm_setting <- list(
  link = "inverse.tangent",
  hyper = list(
    kappa = list(
      initial = log(50),
      prior = "pc.vminf",
      param = c(0.5, 0.5),
      fixed = FALSE
    )
  )
)

direct_data <- list(
  angle_lavm = INLA::inla.mdata(angle),
  z = z
)

fit_direct <- INLA::inla(
  angle_lavm ~ z,
  family = "cloglike",
  data = direct_data,
```

```
  control.family = list(
    cloglike = lavm.cloglike(lavm_setting)
  ),
  control.inla = list(
    cmin = 0,
    compute.initial.values = TRUE
  ),
  control.compute = list(cpo = TRUE, dic = TRUE, waic = TRUE),
  verbose = FALSE
)
```

Do not replace family = "cloglike" by family = "lavm" in this namespaced call. Some INLA versions have an experimental native family with that name, but its links, hyperparameter names, and priors are different from the INLAcircular LAvM interface. The custom prior names pc.vm0 and pc.vminf are implemented inside this package's cloglike.

5.1 LAvM likelihood controls

lavm.cloglike(family.setting) accepts the following controls:

- link is "inverse.tangent" (the default), "scaled.logit", or "scaled.probit".
- hyper*kappa*initial is one finite initial value for $\theta = \log(\kappa)$. The default is 6; it is not a natural-scale concentration.
- hyper*kappa*prior is "pc.vminf" (the default) or "pc.vm0".
- hyper*kappa*param = c(u, alpha) contains two values strictly between zero and one. Here u is a threshold for the mean resultant length $A(\kappa) = I_1(\kappa)/I_0(\kappa)$, and the calibration is $\Pr\{A(\kappa) > u\} = \alpha$.
- hyper*kappa*fixed = TRUE fixes $\log(\kappa)$ at initial; the default is FALSE.

If κ_u solves $A(\kappa_u) = u$, pc.vm0 uses

$$\lambda = -\frac{\log(\alpha)}{d_0(\kappa_u)},$$

whereas pc.vminf uses

$$\lambda = -\frac{\log(1-\alpha)}{\sqrt{1-u}}.$$

This c(u, alpha) interface differs from the standalone d/p/q/r functions, which take lambda directly. The obsolete names hyper$prec and log.initial are rejected.

For pc.vminf, this calibration comes from the full PC law, including its atom at kappa = 0. A fitted INLA hyperparameter is finite $\theta = \log(\kappa)$, so the cloglike evaluates only the continuous component transformed to the θ scale, including its Jacobian. It has no separate discrete state at $\theta = -\infty$. Treat c(u, alpha) as the rate calibration for the fitted continuous prior, not as a claim that the fitted hyperparameter retains the boundary atom.

For numerical stability, the compiled cloglike evaluates its likelihood and prior after restricting the working value of θ to $[-30, 30]$. This matters only at extreme concentrations, but is relevant when selecting initial values or diagnosing an optimizer at a boundary.

The literal lower-level fit retains ordinary INLA cloglike names and the internal $\log(\kappa)$ marginal. Transform it when needed:

```
theta_name <- grep(
  "^Theta1 for",
```

```
  names(fit_direct$marginals.hyperpar),
  value = TRUE
)[1]
kappa_marginal <- INLA::inla.tmarginal(
  exp,
  fit_direct$marginals.hyperpar[[theta_name]]
)
INLA::inla.zmarginal(kappa_marginal)
```

5.2 Convenience call for a single LAvM model

Loading INLA first and INLAcircular second makes the package's compatibility function available as the unqualified inla(). It accepts family = "lavm", performs the three preparation steps above, and renames the concentration output onto the natural kappa scale.

```
library(INLA)
library(INLAcircular)

fit_short <- inla(
  angle ~ z,
  family = "lavm",
  data = data.frame(angle = angle, z = z),
  control.family = lavm_setting
)
```

Use INLA::inla() for the explicit low-level interface and for ordinary INLA families. Use the unqualified inla() form above only when you intentionally want the package's LAvM compatibility layer.

6 Building model specifications

inlacc() uses a formula-building layer to assemble one or more likelihood blocks into a stacked INLA model. Create one response block with

```
likelihood(formula, family, family.setting = NULL)
```

Its arguments are:

- formula: a two-sided formula for one response. The left side should be a single column name in data.
- family: one INLA likelihood name. Use "lavm" for the package's link-adjusted von Mises likelihood; other names are passed to INLA.
- family.setting: the control.family list for this block. For LAvM it has the structure documented in the preceding section.

A single likelihood object can be passed directly to inlacc(). A joint model is a list. Likelihood order determines response-block order and, where relevant, multivariate component order.

```
angle_model <- likelihood(
  angle ~
    intercept(name = "alpha0", mean = 0, sd = 2) +
    covariate(z, name = "alpha_z", mean = 0, sd = 1) +
    f(
      hour_effect,
      model = "rw1",
      cyclic = TRUE,
      constr = TRUE,
      scale.model = TRUE,
```

```
      hyper = list(
        prec = list(
          prior = "pc.prec",
          param = c(1, 0.01)
        )
      )
    ),
  family = "lavm",
  family.setting = lavm_setting
)
```

6.1 Formula terms and their interpretation

The right side may contain ordinary numeric fixed-effect expressions and the following markers:

Term	Meaning inside `inlacc()`
`1`	Automatic block-specific intercept named `Intercept_<response>`
`0` or `-1`	Remove the automatic intercept
`intercept(...)`	Named intercept with an optional Gaussian prior
ordinary term such as `z` or `log(z)`	Block-specific fixed effect evaluated in `data`
`covariate(...)`	Fixed effect or automatically/explicitly copied response predictor
`f(...)`	Standard INLA latent-effect term, passed into the assembled formula

Use numeric columns and expressions for ordinary terms. The assembler does not reproduce every model.matrix() feature for factors, offsets, or special response expressions. Use unique custom coefficient names across likelihoods unless a shared fixed coefficient is intentional.

6.1.1 `intercept()`

```
intercept(name = NULL, mean = NULL, sd = NULL)
```

- name changes the coefficient label; its default is Intercept_<response>.
- mean and sd set the Gaussian prior. A missing value inherits the corresponding global fixed-effect setting.
- A formula 0 or -1 removes the automatic intercept, while an explicit intercept() adds the custom one.

This is a formula marker. It returns NULL when called by itself because inlacc() interprets its unevaluated call.

6.1.2 `covariate()`

```
covariate(
  var,
  name = NULL,
  mean = NULL,
  sd = NULL,
  initial = NULL,
  fixed = FALSE,
  predictor = NULL
)
```

- var is an unquoted variable or expression evaluated in data. It is an argument, not a separate function named var().
- name supplies a stable coefficient and display name.
- mean and sd independently override the Gaussian prior for an ordinary fixed effect.
- predictor = NULL, the default, copies the latent predictor when var is an untransformed response in another likelihood block; otherwise it creates an ordinary observed fixed effect.
- predictor = TRUE explicitly requests copying. It is an error if var is not an untransformed response from another block.
- predictor = FALSE always creates an ordinary fixed effect from the observed values, including when var is another response.
- initial and fixed control the INLA copy-scaling coefficient. They apply to copied predictors and are not used for ordinary fixed effects.

For a copied predictor, provide both mean and sd to customize its scaling prior. If either is absent, the current implementation uses its default $N(0, 31.62^2)$ scaling prior. Any response used directly as a bare term on another block's right side is also copied automatically. To use that response's observed values instead, write covariate(response, predictor = FALSE). A transformed expression such as log(response) is an ordinary observed fixed effect under predictor = NULL or FALSE; requesting predictor = TRUE for a transformed expression is an error because this interface can copy only a named source predictor.

6.1.3 `f()` and standard INLA options

INLAcircular does not export a separate f(). Write a bare f(...) in an inlacc() formula. Its syntax is the R-INLA latent-effect syntax, including model, hyper, constr, scale.model, cyclic, group, control.group, replicate, values, and copy. Use INLA::inla.doc() for details of the selected latent model.

The term is preserved in the assembled INLA formula. INLAcircular additionally does the following:

- supplies an index column from data or index();
- creates 1:nrow(data) when an index is otherwise absent;
- injects or replaces values for the matching first index when a complete process.id is given;
- records common model and hyperprior information for summaries; and
- rewrites the special model = "iidkd_LKJ" form described later.

Because the preprocessor recognizes the call name literally, use f(...), not INLA::f(...), in these formulas.

6.2 Mapping observations with `index()`

```
index(var, data.id, process.id = NULL)
```

Here data.id and process.id are arguments of index(), not separate functions.

- var is a character string matching the first argument of an f() term.
- data.id maps each observation row to a latent-process node and should normally have nrow(data) entries.
- process.id optionally gives the complete ordered node set. When supplied, it becomes the INLA values argument for an f() term whose first argument matches var, replacing any existing values argument. When it is NULL, INLA infers represented values from the observation index. A mapping can supply data.id for a first, group, or replicate variable, but process.id applies only to a matching first index.

```
hour_index <- index(
  var = "hour_effect",
  data.id = c(1, 2, 2, 4),
  process.id = 1:4
)

hour_index
```

```
#> $var
#> [1] "hour_effect"
#>
#> $data.id
#> [1] 1 2 2 4
#>
#> $process.id
#> [1] 1 2 3 4
```

Pass one object directly or several in a list:

```
latent_indices <- list(
  index("hour_effect", data.id = data$hour, process.id = 0:23),
  index("region_effect", data.id = data$region, process.id = region_ids)
)
```

If a mapped name is not used by any f() first, group, or replicate argument, it is ignored with a warning. For an ordinary shared effect, index(var, data.id = "likelihood") sets its index to the likelihood-block number. A later section documents the different special LKJ form process.id = "likelihood".

7 Fitting with `inlacc()`

The complete call is

```
inlacc(
  model,
  data,
  latent.index = NULL,
  LKJ.eta = 5,
  control.family = NULL,
  control.fixed = NULL,
  control.inla = list(cmin = 0, compute.initial.values = TRUE),
  metrics = FALSE,
  verbose = FALSE,
  ...
)
```

7.1 Arguments and defaults

- model is one likelihood() object or a non-empty list of them.
- data is a data frame containing every simple response column, fixed-effect variable, and ordinary latent index. All likelihood blocks use the same number and ordering of observation rows; missing responses are represented with NA and handled by INLA.
- latent.index is one index() object or a list of them.
- LKJ.eta is one positive finite LKJ shape shared by every iidkd_LKJ group. It defaults to 5 and is irrelevant when no such group is present.
- control.family optionally overrides block-level family.setting. Supply one nested control list per likelihood, even for a single block. When no override exists for a block, that block's family.setting is used.
- control.fixed is an INLA fixed-effect control list. The package defaults are mean 0 and precision 0.001, equivalent to SD about 31.62. Its mean and prec may be scalars or INLA named lists with a default entry; existing named entries are preserved. Priors specified in intercept() or covariate() override the matching coefficients.
- control.inla is passed to INLA::inla(). Its package default sets cmin = 0 and compute.initial.values = TRUE. Supplying another list replaces that whole default, so include either setting explicitly when it remains required.

- metrics = TRUE requests CPO, DIC, WAIC, and INLA configuration output. The default is FALSE.
- verbose is passed directly to INLA::inla().
- ... accepts further INLA arguments. Common examples are control.predictor, control.compute, control.mode, num.threads, and blas.num.threads. A supplied control.compute is merged field by field with the values implied by metrics.

Do not pass formula or family through ...: those names would replace the internally assembled values. Use the formal control.family and control.fixed arguments only for their documented controls; the stacked formula and data are owned by inlacc().

7.2 Single-likelihood fit

```
fit <- inlacc(
  model = angle_model,
  data = data.frame(
    angle = angle,
    z = z,
    hour_effect = hour
  ),
  latent.index = index(
    "hour_effect",
    data.id = hour,
    process.id = 0:23
  ),
  control.predictor = list(compute = TRUE),
  metrics = TRUE
)
```

7.3 Joint likelihoods and copied predictors

The following outline fits a circular response and a Gaussian response. The second block uses the first block's latent predictor through an INLA copy term. Because angle is the response of another likelihood block, covariate(angle) selects this behavior automatically.

```
joint_model <- list(
  likelihood(
    angle ~
      intercept(name = "alpha0", mean = 0, sd = 2) +
      covariate(z, name = "alpha_z", mean = 0, sd = 1),
    family = "lavm",
    family.setting = lavm_setting
  ),
  likelihood(
    y ~
      intercept(name = "beta0", mean = 0, sd = 5) +
      covariate(z, name = "beta_z", mean = 0, sd = 2) +
      covariate(
        angle,
        name = "beta_angle_predictor",
        mean = 0,
        sd = 1,
        initial = 0,
        fixed = FALSE
      ),
    family = "gaussian",
```

```
    family.setting = list(
      hyper = list(
        prec = list(prior = "pc.prec", param = c(1, 0.01))
      )
    )
  )
)

joint_fit <- inlacc(
  model = joint_model,
  data = data.frame(angle = angle, y = y, z = z),
  control.predictor = list(compute = TRUE),
  metrics = TRUE
)
```

Writing predictor = TRUE in that term is equivalent but makes the copy request explicit. To regress on the observed angles instead of their latent predictor, use covariate(angle, predictor = FALSE).

Set control.predictor = list(compute = TRUE) when fitted linear predictors are needed. The custom summary warns when predictor computation is particularly relevant, including models with copied covariates.

7.4 Returned object

The result inherits from both inlacc and INLA result classes, so ordinary INLA components remain available. The package also records:

- the assembled formula, stacked data, effective families, and family controls in inlacc_meta;
- fixed-effect, hyperparameter, prior, and latent-process mappings used by summary.inlacc();
- the names of copied predictors; and
- lkj_modes and inlacc_meta$LKJ, both always lists and empty when no LKJ effect is present.

LAvM concentration marginals are normally transformed from $\log(\kappa)$ to κ and renamed in the fitted object. Raw INLA fields such as summary.fixed, summary.hyperpar, marginals.fixed, marginals.hyperpar, summary.fitted.values, and cpu.used remain available.

8 Modelling multivariate effects with graphpcor

INLAcircular integrates one specific graphpcor model: a dense LKJ-correlated IID vector shared across likelihood blocks. It does not expose graphpcor's general graph-structured or tree-structured model interfaces.

Install the optional dependencies before using this feature:

```
install.packages(c("graphpcor", "INLAtools"))
```

The feature inherits graphpcor's own R-version and dependency requirements. Both packages are checked at run time.

8.1 Declaring an LKJ-correlated effect

Use the exact special syntax

```
f(
  i,
  model = "iidkd_LKJ",
  pc.prior.u = 1,
  pc.prior.alpha = 0.5
)
```

in at least two likelihood blocks. Terms with the same bare first argument (i above) form one multivariate process. Component order follows the likelihood objects supplied to inlacc().

- A process may occur at most once in each likelihood.
- Its dimension is the number of likelihoods containing the term and must be at least two.
- The same index name cannot also be used by another f() model.
- LKJ.eta is supplied once to inlacc(), not as eta inside f().
- prior.param is not supported for this special term.

The per-component scale prior is passed to graphpcor as sigma.prior.reference = pc.prior.u and sigma.prior.probability = pc.prior.alpha. The defaults are 1 and 0.5. The reference must be positive; the probability may be in [0, 1] or NA. The PC-scale calibration used by the implementation is $\Pr(\sigma > u) = \alpha$, equivalently $\lambda = -\log(\alpha)/u$. Values 0, 1, and NA fix that component's standard deviation at u rather than estimating a scale parameter.

```
multivariate_model <- list(
  likelihood(
    angle ~ intercept(name = "alpha0") +
      f(
        i,
        model = "iidkd_LKJ",
        pc.prior.u = 1,
        pc.prior.alpha = 0.5
      ),
    family = "lavm",
    family.setting = lavm_setting
  ),
  likelihood(
    y ~ intercept(name = "beta0") +
      f(
        i,
        model = "iidkd_LKJ",
        pc.prior.u = 2,
        pc.prior.alpha = 0.5
      ),
    family = "gaussian"
  )
)

fit_lkj <- inlacc(
  multivariate_model,
  data = data.frame(angle = angle, y = y),
  LKJ.eta = 5,
  control.predictor = list(compute = TRUE)
)
```

LKJcc() performs the preprocessing internally: it collects the terms, constructs graphpcor's cgeneric model, rewrites the formulas, and creates component and replicate indices. Most users should not call it directly. Its return value contains the rewritten formulas, generated models, block data, process metadata, and unconsumed ordinary index mappings.

8.2 Replicate indices

By default, observations receive replicate IDs seq_len(nrow(data)); no i column is required. To use explicit replicate IDs, supply the special mapping

```
lkj_index <- index(
  var = "i",
  data.id = replicate_id,
  process.id = "likelihood"
)

fit_lkj <- inlacc(
  multivariate_model,
  data = data,
  latent.index = lkj_index,
  LKJ.eta = 5
)
```

In this special form, data.id must contain one positive integer per observation and becomes the INLA replicate index. The component index is generated from likelihood order. An explicit replicate = some_name must use a bare variable name. It may be written in one or more special f() terms, but every explicit name within a group must agree. That name controls the generated replicate column; its values still come from this mapping or the default row sequence.

8.3 Recovering covariance information

fit_lkj$lkj_modes[["i"]] stores internal posterior-mode values, not a full posterior correlation summary. When every scale is free ($0 <$ pc.prior.alpha < 1), the first p entries for a p-dimensional group are log standard deviations and the remaining $p(p-1)/2$ entries parameterize the correlation matrix. The example above has this all-free form.

```
p <- 2L
theta_i <- fit_lkj$lkj_modes[["i"]]

sd_mode <- exp(theta_i[seq_len(p)])
correlation_mode <- graphpcor::basecor(
  theta_i[-seq_len(p)],
  p = p
)$base

sd_mode
correlation_mode
```

For a mixture of free and fixed scales, reconstruct the complete SD vector before extracting the correlation parameters:

```
process_i <- fit_lkj$inlacc_meta$LKJ[["i"]]
u <- process_i$pc.prior.u
alpha <- process_i$pc.prior.alpha
fixed_scale <- is.na(alpha) | alpha <= 0 | alpha >= 1
n_free <- sum(!fixed_scale)

theta_i <- fit_lkj$lkj_modes[["i"]]
log_sd_mode <- log(u)
log_sd_mode[!fixed_scale] <- theta_i[seq_len(n_free)]

n_cor <- p * (p - 1L) / 2L
cor_index <- n_free + seq_len(n_cor)
correlation_mode <- graphpcor::basecor(
  theta_i[cor_index],
```

```
  p = p
)$base

sd_mode <- exp(log_sd_mode)
```

For multiple LKJ groups, use the corresponding first-argument name as the list key. Component order is always the likelihood order used to build the group. These are point estimates at the internal posterior mode; use INLA's marginal or sampling tools when full uncertainty summaries are required.

9 Results and summaries

summary() dispatches to summary.inlacc() for fitted models:

```
summary(object, decimal = 3, ...)
```

It returns an inlacc_summary object with the following components:

- fixed_par: fixed-effect posterior summaries and displayed Gaussian priors;
- likelihood_par: transformed likelihood parameters, including natural-scale LAvM concentration and Gaussian standard deviations;
- random_par: latent-process hyperparameters and their prior labels;
- time.summary: setup, computation, post-processing, and total times;
- metrics: available marginal likelihood, DIC, and WAIC summaries; and
- needs_pred_warning: whether the fitted model would benefit from control.predictor = list(compute = TRUE).

The decimal value controls printing, not stored posterior precision. print.inlacc_summary(x, decimal = NULL, metrics = TRUE) can override the number of displayed decimal places and suppress the metrics section. Timing components use print.inlacc_time(x). These methods are normally called through summary() and print() rather than directly.

```
fit_summary <- summary(fit, decimal = 3)
print(fit_summary, metrics = TRUE)
```

Because an inlacc result remains an INLA result, the original posterior summaries, marginals, fitted values, predictor values, model criteria, and timing fields remain accessible with standard INLA component names. CPO values remain on the raw fit even though the custom printed summary does not create a separate CPO table.

10 Utilities and included data

10.1 Modified Bessel function

bessel_i(x, nu, expon.scaled = FALSE) evaluates the modified Bessel function of the first kind using C routines adapted from GSL. R is only the interface. The order nu must be finite and nonnegative; fractional orders are supported for nonnegative x. Negative x is supported for integer orders, with the usual parity rule; otherwise it produces NaN and a warning. Set expon.scaled = TRUE to return $I_\nu(x)\exp(-|x|)$, which avoids overflow for large x. Very large orders can reduce the accuracy of unscaled values near the transition between exponential growth and decay; such inputs produce an accuracy warning when the estimated loss is significant.

```
bessel_i(c(0, 1, 5), nu = 0)
#> [1]  1.000000  1.266066 27.239872
bessel_i(1, nu = 0.5)
#> [1] 0.9376749
```

```r
bessel_i(1000, nu = 0, expon.scaled = TRUE)
#> [1] 0.01261724
```

10.2 New York wind data

wind_newyork contains 8,759 hourly observations from JFK International Airport, with circular wind direction, wind speed, temperature, timestamp, month, day, and hour fields.

```r
data("wind_newyork", package = "INLAcircular")
dim(wind_newyork)
#> [1] 8759    8
utils::head(names(wind_newyork), 4)
#> [1] "DATE"            "HLY.WIND.VCTDIR" "HLY.WIND.VCTSPD" "HLY.TEMP.NORMAL"
```

11 Complete public API index

Area	Functions
Von Mises distribution	dvm, pvm, qvm, rvm
Link-adjusted von Mises	lavm.link, dlavm, plavm, qlavm, rlavm
Other circular densities	dcardioid (dcard), dwrappedcauchy (dwc)
Von Mises PC prior, uniform base	dpc.vm0, ppc.vm0, qpc.vm0, rpc.vm0
Von Mises PC prior, point-mass base	dpc.vminf, ppc.vminf, qpc.vminf, rpc.vminf
Cardioid PC prior, uniform base	dpc.card0, ppc.card0, qpc.card0, rpc.card0
Cardioid PC prior, kappa = 1/2 base	dpc.card, ppc.card, qpc.card, rpc.card
Wrapped Cauchy PC prior	dpc.wc, ppc.wc, qpc.wc, rpc.wc
Model specification	likelihood, intercept, covariate, index
Model fitting	inlacc, inla, lavm.cloglike
Multivariate preprocessing	LKJcc
Results	summary.inlacc, print.inlacc_summary, print.inlacc_time
Numerical utility	bessel_i
Included data	wind_newyork

Function-level help is available with ?function_name. For names containing dots, quote the topic if needed, for example help("dpc.card0").

12 Numerical implementation notes

The new cardioid and wrapped Cauchy routines use compiled C throughout their numerical path. Endpoint limits, log-density calculations, inverse transforms, and random generation are not implemented as R formulas. Cancellation-prone expressions use algebraic reductions, log1p(), expm1(), and local series near difficult boundaries.

The standalone INLA sources under src-cloglike/ are separate from the R package numerical interface. Installing the R package compiles src/, not src-cloglike/.

13 Reproducibility

```r
R.version.string
#> [1] "R version 4.6.1 (2026-06-24)"
```

```r
packageVersion("INLAcircular")
#> [1] '1.0.0'
```